

Lab - WebSockets

Objetivo

Demonstrar comunicação persistente por WebSockets com um servidor Python e clientes de navegador e terminal. A implementação fornece echo e chat, registra sessões e distribui mensagens para todos os clientes conectados.

Ambiente e execução

Execução em Linux usando uma imagem Python 3.12, com dependências fixadas. No computador Windows, Docker Desktop utiliza o backend WSL2. O README também descreve a execução direta em Linux com ambiente virtual. O servidor escuta nas interfaces do contêiner e a porta é publicada somente no endereço local do computador.

Dependências da aplicação e dos testes

```
websockets==17.1
pytest==9.1.1
pytest-timeout==2.4.0
iniconfig==2.3.0
packaging==26.3
pluggy==1.6.0
Pygments==2.21.0
```

Os comandos abaixo são executados na pasta do projeto.

```
docker compose up --build -d servidor
http://localhost:8080/ui-echo
http://localhost:8080/ui-chat
docker compose exec servidor python -m websockets ws://127.0.0.1:8080/chat
docker compose run --rm testes
docker compose logs --tail 30 servidor
docker compose down
```

Implementação

Os clientes HTML são servidos por HTTP em /ui-echo e /ui-chat. Depois de abrir a página, o navegador estabelece uma conexão persistente em /echo ou /chat. O echo devolve a mensagem somente ao remetente; o chat a envia a todas as sessões ativas, inclusive a origem.

A função `async` define uma corrotina. Ao executar `await` numa operação de rede, ela permite que o event loop avance no atendimento de outras conexões. O dicionário `sessions` usa a tupla IP e porta do cliente, obtida de `remote_address`, como chave.

O broadcast percorre uma cópia de `sessions.values()`. Assim, uma desconexão durante um `await` não altera o objeto percorrido. Conexões encerradas são tratadas individualmente; `finally` remove a sessão e os demais clientes continuam ativos. Conexão, mensagem e desconexão geram logs com endereço remoto.

A assinatura `chat(websocket, sessions={})` segue o roteiro. Cada instância do servidor passa seu próprio dicionário explicitamente, evitando dependência do argumento padrão mutável. A manutenção das sessões é em memória e não guarda histórico de mensagens.

A base foi adaptada à API atual de `websockets.asyncio.server`. O roteamento usa `request.path` e o tratamento HTTP recebe `connection` e `request`. No enunciado, a URL da demonstração de echo está trocada: a correta é `/ui-echo`. No Docker, o IP remoto pode refletir o gateway virtual; as portas identificam as conexões distintas.

Testes e resultados

A suíte abre servidores reais em portas livres e verifica páginas HTTP, caracteres acentuados, isolamento do echo, broadcast para três clientes, desconexão normal e abrupta, remoção de sessões, reconexão e rejeição de rotas inválidas. Os testes incluem tempos limite e encerram os servidores ao terminar.

Na validação registrada em 22/09/2026 às 17:41 UTC, 5 testes passaram em Linux, com Python 3.12.14, sem falhas, erros ou testes ignorados. O registro completo acompanha a entrega em `validacao.txt`.

Evidências de execução

Capturas reais realizadas no navegador em 23/09/2026 (horário de Brasília). Os terminais usam `ttyd` em um contêiner Linux temporário com Python 3.12.14. Os comandos Python são executados diretamente nesse contêiner; correspondem às chamadas `docker compose` descritas no README. Os horários dos logs e terminais estão em UTC. Dois clientes HTML originais foram abertos em painéis independentes lado a lado, conectados ao servidor pela porta local 18080. O cliente C é a CLI oficial de `websockets` em uma sessão de terminal compartilhada. A mensagem parte de C e retorna aos três clientes. A página do cliente B é fechada antes da segunda mensagem; o log confirma a remoção da sessão e dois clientes ativos.

Figura 1

PG1 — Ambiente e servidor

Execução real em Linux / Docker · terminal web ttyd · 23/09/2026

```
Linux | pg1_websockets | 2026-09-23 21:04:21 UTC

$ python -c "import platform; print(platform.platform()); print(platform.python_version())"
Linux-6.18.33.2-microsoft-standard-WSL2-x86_64-with-glibc2.36
3.12.14

$ head -n 3 /tmp/websocket.log
2026-09-23 21:04:12,105 INFO server listening on 0.0.0.0:8080
2026-09-23 21:04:12,106 INFO Ambiente=Linux Python=3.12.14
2026-09-23 21:04:12,106 INFO Servidor em 0.0.0.0:8080; clientes /ui-echo e /ui-chat

$
```

Ambiente Linux e inicialização do servidor

Figura 2

Echo com WebSockets

Cada mensagem enviada retorna somente para esta conexão.

Estado: **Conectado**

Mensagem

EnviarFecharReconectar

Mensagens recebidas

- Teste de echo: olá!

PG1 · Sistemas Distribuídos · [Abrir chat](#)

Mensagem devolvida pelo serviço de echo

Figura 3

PG1 — Broadcast e desconexão

Execução real em Linux / Docker · terminal web ttyd · 23/09/2026
[Fechar conexão do cliente B](#)

Cliente A — http://localhost:18080/ui-chat

Estado: Conectado

Mensagem

Enviar Fechar Reconectar

Mensagens recebidas

- Cliente C: olá para todos

Cliente B — http://localhost:18080/ui-chat

Estado: Conectado

Mensagem

Enviar Fechar Reconectar

Mensagens recebidas

- Cliente C: olá para todos

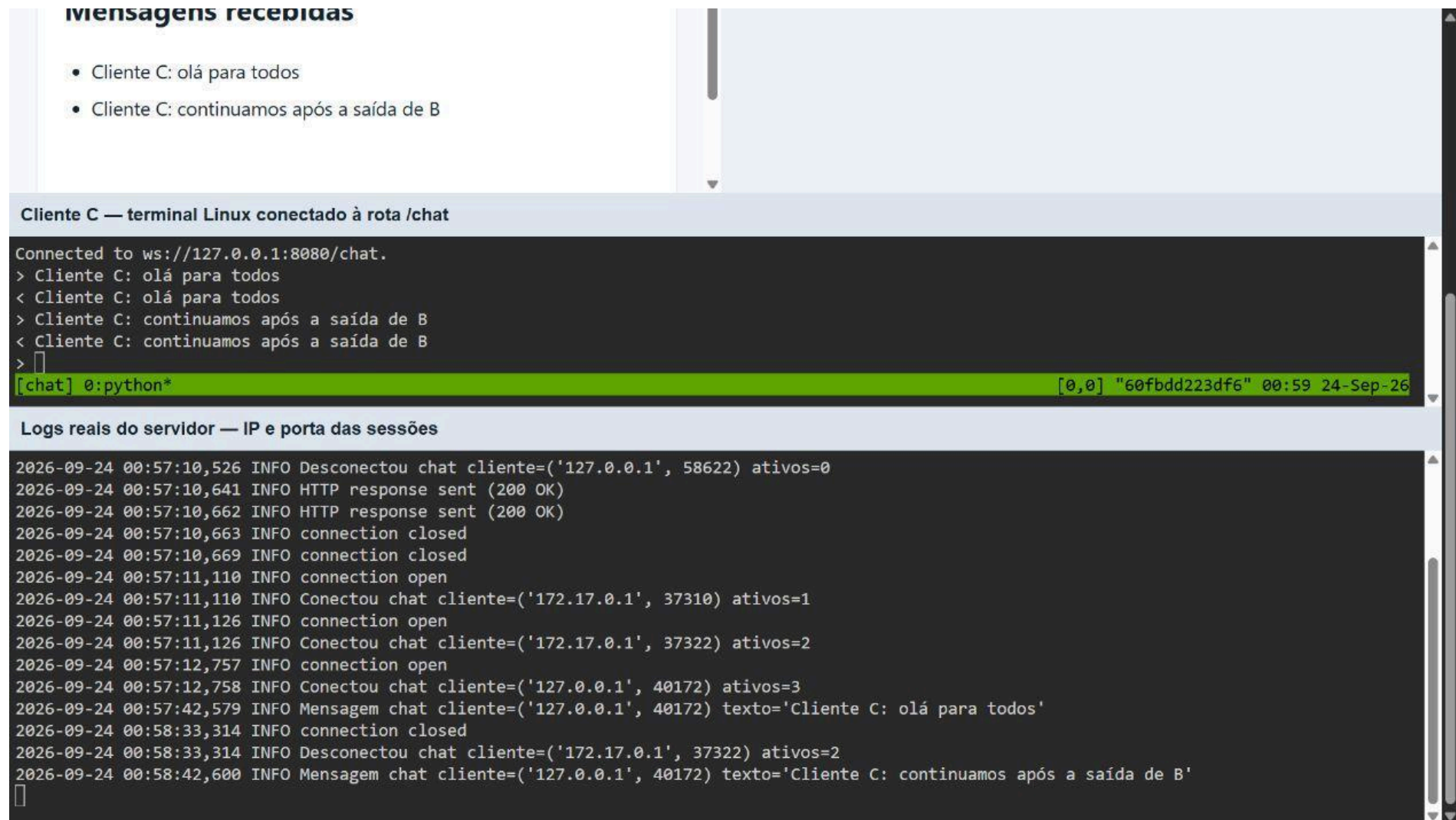
Cliente C — terminal Linux conectado à rota /chat

```
Connected to ws://127.0.0.1:8080/chat.  
> Cliente C: olá para todos  
< Cliente C: olá para todos  
> 
```

[chat] 0:python* [0,0] "60fbdd223df6" 00:58 24-Sep-26

Mensagem do cliente C recebida por A, B e pelo próprio C. Dois clientes HTML independentes e a CLI estão visíveis simultaneamente.

Figura 4



The screenshot displays a web application interface. At the top, a section titled "mensagens recebidas" (received messages) contains a list of messages from "Cliente C": "olá para todos" and "continuamos após a saída de B". Below this is a terminal window titled "Cliente C — terminal Linux conectado à rota /chat". The terminal shows a successful connection to "ws://127.0.0.1:8080/chat" and the same two messages from "Cliente C". A green status bar at the bottom of the terminal indicates "[chat] 0:python*" and "[0,0] '60fbdd223df6' 00:59 24-Sep-26". Below the terminal is a section titled "Logs reais do servidor — IP e porta das sessões" (real server logs — IP and port of sessions). This section contains a log of server events, including connection status changes and messages received from clients. The log shows that after a client disconnects, the active session count decreases, and when a new client connects, it increases. The final log entry shows a message received from "Cliente C" with the text "continuamos após a saída de B".

mensagens recebidas

- Cliente C: olá para todos
- Cliente C: continuamos após a saída de B

Cliente C — terminal Linux conectado à rota /chat

```
Connected to ws://127.0.0.1:8080/chat.  
> Cliente C: olá para todos  
< Cliente C: olá para todos  
> Cliente C: continuamos após a saída de B  
< Cliente C: continuamos após a saída de B  
>   
[chat] 0:python* [0,0] "60fbdd223df6" 00:59 24-Sep-26
```

Logs reais do servidor — IP e porta das sessões

```
2026-09-24 00:57:10,526 INFO Desconectou chat cliente=('127.0.0.1', 58622) ativos=0  
2026-09-24 00:57:10,641 INFO HTTP response sent (200 OK)  
2026-09-24 00:57:10,662 INFO HTTP response sent (200 OK)  
2026-09-24 00:57:10,663 INFO connection closed  
2026-09-24 00:57:10,669 INFO connection closed  
2026-09-24 00:57:11,110 INFO connection open  
2026-09-24 00:57:11,110 INFO Conectou chat cliente=('172.17.0.1', 37310) ativos=1  
2026-09-24 00:57:11,126 INFO connection open  
2026-09-24 00:57:11,126 INFO Conectou chat cliente=('172.17.0.1', 37322) ativos=2  
2026-09-24 00:57:12,757 INFO connection open  
2026-09-24 00:57:12,758 INFO Conectou chat cliente=('127.0.0.1', 40172) ativos=3  
2026-09-24 00:57:42,579 INFO Mensagem chat cliente=('127.0.0.1', 40172) texto='Cliente C: olá para todos'  
2026-09-24 00:58:33,314 INFO connection closed  
2026-09-24 00:58:33,314 INFO Desconectou chat cliente=('172.17.0.1', 37322) ativos=2  
2026-09-24 00:58:42,600 INFO Mensagem chat cliente=('127.0.0.1', 40172) texto='Cliente C: continuamos após a saída de B'  
  

```

Após fechar a página B, A e C continuam recebendo mensagens. O log registra IP/porta da sessão removida e ativos=2.

Figura 5

PG1 — Testes de integração

Execução real em Linux / Docker · terminal web ttyd · 23/09/2026

```
Linux | pg1_websockets | 2026-09-24 00:57:18 UTC

$ python -m pytest -v --color=yes
===== test session starts =====
platform linux -- Python 3.12.14, pytest-9.1.1, pluggy-1.6.0 -- /usr/local/bin/python
rootdir: /labs/pg1_websockets
configfile: pytest.ini
plugins: timeout-2.4.0
timeout: 15.0s
timeout method: signal
timeout func_only: False
collected 5 items

test_websocket.py::test_http_pages_and_missing_route PASSED [ 20%]
test_websocket.py::test_echo_isolated_from_other_echo_and_chat PASSED [ 40%]
test_websocket.py::test_broadcast_three_clients_disconnect_and_reconnect PASSED [ 60%]
test_websocket.py::test_abrupt_disconnect_does_not_break_broadcast PASSED [ 80%]
test_websocket.py::test_invalid_websocket_route_and_server_remains_available PASSED [100%]

===== 5 passed in 1.31s =====

$ █
```

Resultado dos testes automatizados

Referências

Enunciado da atividade: enunciado.pdf. Documentação oficial: <https://websockets.readthedocs.io/>

Procedimentos locais de execução: README.md.

Código fonte desenvolvido

As listagens seguintes contêm a implementação do laboratório e seus testes. As quebras de linhas longas são apenas de apresentação. Os arquivos executáveis originais acompanham a entrega.

websocket.py

```
"""Echo e chat derivados da implementação-base PG1-WS da disciplina."""

import argparse
import asyncio
import logging
import platform
from http import HTTPStatus
from pathlib import Path
from urllib.parse import urlsplit

from websockets.asyncio.server import serve
from websockets.exceptions import ConnectionClosed

LOGGER = logging.getLogger("pg1")
ROOT = Path(__file__).resolve().parent


def http_handler(connection, request):
    """Sirva os clientes HTML sem interferir no handshake WebSocket."""
    path = urlsplit(request.path).path
    pages = {"/ui-chat": "chat.html", "/ui-echo": "echo.html"}
    if path in pages:
        response = connection.respond(
            HTTPStatus.OK, (ROOT / pages[path]).read_text(encoding="utf-8")
        )
        del response.headers["Content-Type"]
        response.headers["Content-Type"] = "text/html; charset=utf-8"
        return response
    if request.headers.get("Upgrade", "").lower() == "websocket":
        return None
    return connection.respond(HTTPStatus.NOT_FOUND, "Página não encontrada.\n")


async def echo(websocket):
    remote = websocket.remote_address
    LOGGER.info("Conectou echo cliente=%s", remote)
    try:
        async for message in websocket:
            LOGGER.info("Mensagem echo cliente=%s texto=%r", remote, message)
            await websocket.send(message)
    except ConnectionClosed:
        pass
    finally:
        LOGGER.info("Desconectou echo cliente=%s", remote)


async def chat(websocket, sessions={}):
    # Assinatura solicitada pelo roteiro. create_server passa um dict próprio
    # explicitamente, evitando compartilhar o valor padrão entre servidores.
    remote = websocket.remote_address
    sessions[remote] = websocket
    LOGGER.info("Conectou chat cliente=%s ativos=%d", remote, len(sessions))
    try:
        async for message in websocket:
            LOGGER.info("Mensagem chat cliente=%s texto=%r", remote, message)
            # A sessão pode ser removida durante um await; itere uma cópia.
            for socket in list(sessions.values()):
```

```

        try:
            await socket.send(message)
        except ConnectionClosed:
            # O finally do handler desse cliente remove sua sessão.
            continue
    except ConnectionClosed:
        pass
    finally:
        if sessions.get(remote) is websocket:
            sessions.pop(remote)
        LOGGER.info("Desconectou chat cliente=%s ativos=%d", remote, len(sessions))

async def web_socket_router(websocket, sessions):
    path = urlsplit(websocket.request.path).path
    if path == "/echo":
        await echo(websocket)
    elif path == "/chat":
        await chat(websocket, sessions)
    else:
        await websocket.close(code=1008, reason="Rota WebSocket inexistente")

def create_server(host="127.0.0.1", port=8080, sessions=None):
    """Uma tabela de sessões por servidor; port=0 permite testes sem colisão."""
    if sessions is None:
        sessions = {}

    async def router(websocket):
        await web_socket_router(websocket, sessions)

    return serve(router, host, port, process_request=http_handler)

async def main(host, port):
    async with create_server(host, port):
        LOGGER.info("Ambiente=%s Python=%s", platform.system(),
platform.python_version())
        LOGGER.info("Servidor em %s:%d; clientes /ui-echo e /ui-chat", host, port)
        await asyncio.Future()

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Servidor de echo e chat WebSocket")
    parser.add_argument("--host", default="127.0.0.1")
    parser.add_argument("--port", type=int, default=8080)
    args = parser.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s
%(message)s")
    try:
        asyncio.run(main(args.host, args.port))
    except KeyboardInterrupt:
        LOGGER.info("Servidor encerrado")

```

chat.html

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Chat | PG1 WebSockets</title>
    <style>
        body { font: 17px system-ui, sans-serif; margin: 40px auto; padding: 0 24px;
max-width: 760px; color: #18252f; background: #f7f9fb; }
        main { background: white; border: 1px solid #d7dfe5; border-radius: 8px; padding:
24px; }
        h1 { margin: 0 0 12px; }
        label { display: block; margin-bottom: 6px; }
        input { width: 95%; padding: 10px; font: inherit; margin-bottom: 12px; }
        button { font: inherit; padding: 8px 14px; margin: 0 8px 8px 0; cursor: pointer; }
        button:disabled { cursor: default; }
        #area { min-height: 140px; padding-left: 24px; overflow-wrap: anywhere; }
        li { padding: 5px 0; white-space: pre-wrap; }
    </style>

```

```

    #status { font-weight: 600; }
    small { color: #455a64; }
  </style>
</head>
<body>
<main>
  <h1>Chat com WebSockets</h1>
  <p>Abra esta página em duas janelas e um cliente de terminal. Todos recebem cada
mensagem.</p>
  <p>Estado: <span id="status" role="status">Conectando...</span></p>
  <form id="form">
    <label for="msg">Mensagem</label>
    <input id="msg" type="text" autocomplete="off" placeholder="Digite sua mensagem">
    <button id="enviar" type="submit" disabled>Enviar</button>
    <button id="close" type="button" disabled>Fechar</button>
    <button id="reconnect" type="button" disabled>Reconectar</button>
  </form>
  <h2>Mensagens recebidas</h2>
  <ul id="area" aria-live="polite"></ul>
  <small>PG1 · Sistemas Distribuídos · <a href="/ui-echo">Abrir echo</a></small>
</main>
<script>
const area = document.getElementById("area");
const msg = document.getElementById("msg");
const status = document.getElementById("status");
const enviar = document.getElementById("enviar");
const fechar = document.getElementById("close");
const reconectar = document.getElementById("reconnect");
let ws;

function conectar() {
  status.textContent = "Conectando...";
  enviar.disabled = fechar.disabled = reconectar.disabled = true;
  const protocol = location.protocol === "https:" ? "wss:" : "ws:";
  ws = new WebSocket(protocol + "://" + location.host + "/chat");
  ws.onopen = () => {
    status.textContent = "Conectado";
    enviar.disabled = fechar.disabled = false;
    msg.focus();
  };
  ws.onmessage = (event) => {
    const entry = document.createElement("li");
    entry.textContent = event.data;
    area.appendChild(entry);
  };
  ws.onclose = () => {
    status.textContent = "Desconectado";
    enviar.disabled = fechar.disabled = true;
    reconectar.disabled = false;
  };
  ws.onerror = () => { status.textContent = "Falha na conexão. Verifique o servidor." };
};
document.getElementById("form").onsubmit = (event) => {
  event.preventDefault();
  if (ws.readyState !== WebSocket.OPEN || !msg.value.trim()) return;
  ws.send(msg.value);
  msg.value = "";
  msg.focus();
};
fechar.onclick = () => ws.close();
reconectar.onclick = conectar;
conectar();
</script>
</body>
</html>

```

echo.html

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">

```

```

<title>Echo | PG1 WebSockets</title>
<style>
  body { font: 17px system-ui, sans-serif; margin: 40px auto; padding: 0 24px;
max-width: 760px; color: #18252f; background: #f7f9fb; }
  main { background: white; border: 1px solid #d7dfe5; border-radius: 8px; padding:
24px; }
  h1 { margin: 0 0 12px; }
  label { display: block; margin-bottom: 6px; }
  input { width: 95%; padding: 10px; font: inherit; margin-bottom: 12px; }
  button { font: inherit; padding: 8px 14px; margin: 0 8px 8px 0; cursor: pointer; }
  button:disabled { cursor: default; }
  #area { min-height: 140px; padding-left: 24px; overflow-wrap: anywhere; }
  li { padding: 5px 0; white-space: pre-wrap; }
  #status { font-weight: 600; }
  small { color: #455a64; }
</style>
</head>
<body>
<main>
  <h1>Echo com WebSockets</h1>
  <p>Cada mensagem enviada retorna somente para esta conexão.</p>
  <p>Estado: <span id="status" role="status">Conectando...</span></p>
  <form id="form">
    <label for="msg">Mensagem</label>
    <input id="msg" type="text" autocomplete="off" placeholder="Digite sua mensagem">
    <button id="enviar" type="submit" disabled>Enviar</button>
    <button id="close" type="button" disabled>Fechar</button>
    <button id="reconnect" type="button" disabled>Reconectar</button>
  </form>
  <h2>Mensagens recebidas</h2>
  <ul id="area" aria-live="polite"></ul>
  <small>PG1 · Sistemas Distribuídos · <a href="/ui-chat">Abrir chat</a></small>
</main>
<script>
  const area = document.getElementById("area");
  const msg = document.getElementById("msg");
  const status = document.getElementById("status");
  const enviar = document.getElementById("enviar");
  const fechar = document.getElementById("close");
  const reconectar = document.getElementById("reconnect");
  let ws;

  function conectar() {
    status.textContent = "Conectando...";
    enviar.disabled = fechar.disabled = reconectar.disabled = true;
    const protocol = location.protocol === "https:" ? "wss:" : "ws:";
    ws = new WebSocket(protocol + "://" + location.host + "/echo");
    ws.onopen = () => {
      status.textContent = "Conectado";
      enviar.disabled = fechar.disabled = false;
      msg.focus();
    };
    ws.onmessage = (event) => {
      const entry = document.createElement("li");
      entry.textContent = event.data;
      area.appendChild(entry);
    };
    ws.onclose = () => {
      status.textContent = "Desconectado";
      enviar.disabled = fechar.disabled = true;
      reconectar.disabled = false;
    };
    ws.onerror = () => { status.textContent = "Falha na conexão. Verifique o servidor." };
  };
  document.getElementById("form").onsubmit = (event) => {
    event.preventDefault();
    if (ws.readyState !== WebSocket.OPEN || !msg.value.trim()) return;
    ws.send(msg.value);
    msg.value = "";
    msg.focus();
  };
  fechar.onclick = () => ws.close();
  reconectar.onclick = conectar;

```

```

    conectar();
</script>
</body>
</html>

```

test_websocket.py

```

"""Integração por HTTP e WebSocket reais, em portas livres."""

```

```

import asyncio
import logging
from contextlib import asynccontextmanager
from urllib.error import HTTPError
from urllib.request import urlopen

import pytest
from websockets.asyncio.client import connect
from websockets.exceptions import ConnectionClosed

from websocket import create_server

@asynccontextmanager
async def running_server():
    sessions = {}
    async with create_server(port=0, sessions=sessions) as server:
        port = server.sockets[0].getsockname()[1]
        yield f"127.0.0.1:{port}", sessions

async def receive(client):
    return await asyncio.wait_for(client.recv(), timeout=2)

async def wait_sessions(sessions, count):
    async def changed():
        while len(sessions) != count:
            await asyncio.sleep(0.01)
    await asyncio.wait_for(changed(), timeout=2)

def test_http_pages_and_missing_route():
    def fetch(url):
        with urlopen(url, timeout=2) as response:
            return response.status, response.headers.get_content_type(),
            response.read().decode()

    async def scenario():
        async with running_server() as (address, _):
            for path, title in (("ui-chat", "Chat"), ("ui-echo", "Echo")):
                status, content_type, body = await asyncio.to_thread(fetch,
                    f"http://{address}{path}")
                assert status == 200
                assert content_type == "text/html"
                assert title in body
                assert "Distribuído" in body
            with pytest.raises(HTTPError) as error:
                await asyncio.to_thread(fetch, f"http://{address}/inexistente")
            assert error.value.code == 404
            error.value.close()
    asyncio.run(scenario())

def test_echo_isolated_from_other_echo_and_chat():
    async def scenario():
        async with running_server() as (address, _):
            async with (
                connect(f"ws://{address}/echo", proxy=None) as sender,
                connect(f"ws://{address}/echo", proxy=None) as other,
                connect(f"ws://{address}/chat", proxy=None) as chat,
            ):
                await sender.send("Olá, ação! 🌍")
                assert await receive(sender) == "Olá, ação! 🌍"
                for client in (other, chat):

```

```

        with pytest.raises(TimeoutError):
            await asyncio.wait_for(client.recv(), timeout=0.15)
    asyncio.run(scenario())

def test_broadcast_three_clients_disconnect_and_reconnect(caplog):
    caplog.set_level(logging.INFO, logger="pg1")

    async def scenario():
        async with running_server() as (address, sessions):
            async with (
                connect(f"ws://{address}/chat", proxy=None) as a,
                connect(f"ws://{address}/chat", proxy=None) as b,
                connect(f"ws://{address}/chat", proxy=None) as c,
            ):
                await wait_sessions(sessions, 3)
                remote_c = c.local_address
                for sender, message in ((a, "A: olá!"), (b, "B: ação e comunicação")):
                    await sender.send(message)
                    assert await asyncio.gather(*(receive(x) for x in (a, b, c))) ==
[message] * 3
                    await c.close()
                    await wait_sessions(sessions, 2)
                    assert remote_c not in sessions
                    await a.send("A: ainda conectados")
                    assert await receive(a) == await receive(b) == "A: ainda conectados"
                    async with connect(f"ws://{address}/chat", proxy=None) as reconnected:
                        await wait_sessions(sessions, 3)
                        await reconnected.send("C: voltei")
                        assert await asyncio.gather(*(receive(x) for x in (a, b,
reconnected))) == ["C: voltei"] * 3
                        await wait_sessions(sessions, 0)
                    asyncio.run(scenario())
                assert "Conectou chat cliente=('127.0.0.1'," in caplog.text
                assert "Desconectou chat cliente=" in caplog.text
                assert "texto='C: voltei'" in caplog.text

def test_abrupt_disconnect_does_not_break_broadcast():
    async def scenario():
        async with running_server() as (address, sessions):
            async with (
                connect(f"ws://{address}/chat", proxy=None) as a,
                connect(f"ws://{address}/chat", proxy=None) as b,
                connect(f"ws://{address}/chat", proxy=None) as c,
            ):
                await wait_sessions(sessions, 3)
                c.transport.abort()
                await a.send("Servidor continua disponível")
                assert await receive(a) == await receive(b) == "Servidor continua
disponível"
                await wait_sessions(sessions, 2)
            asyncio.run(scenario())

def test_invalid_websocket_route_and_server_remains_available():
    async def scenario():
        async with running_server() as (address, _):
            async with connect(f"ws://{address}/inexistente", proxy=None) as client:
                with pytest.raises(ConnectionClosed):
                    await receive(client)
                assert client.close_code == 1008
            async with connect(f"ws://{address}/echo", proxy=None) as client:
                await client.send("ok")
                assert await receive(client) == "ok"
            asyncio.run(scenario())

```